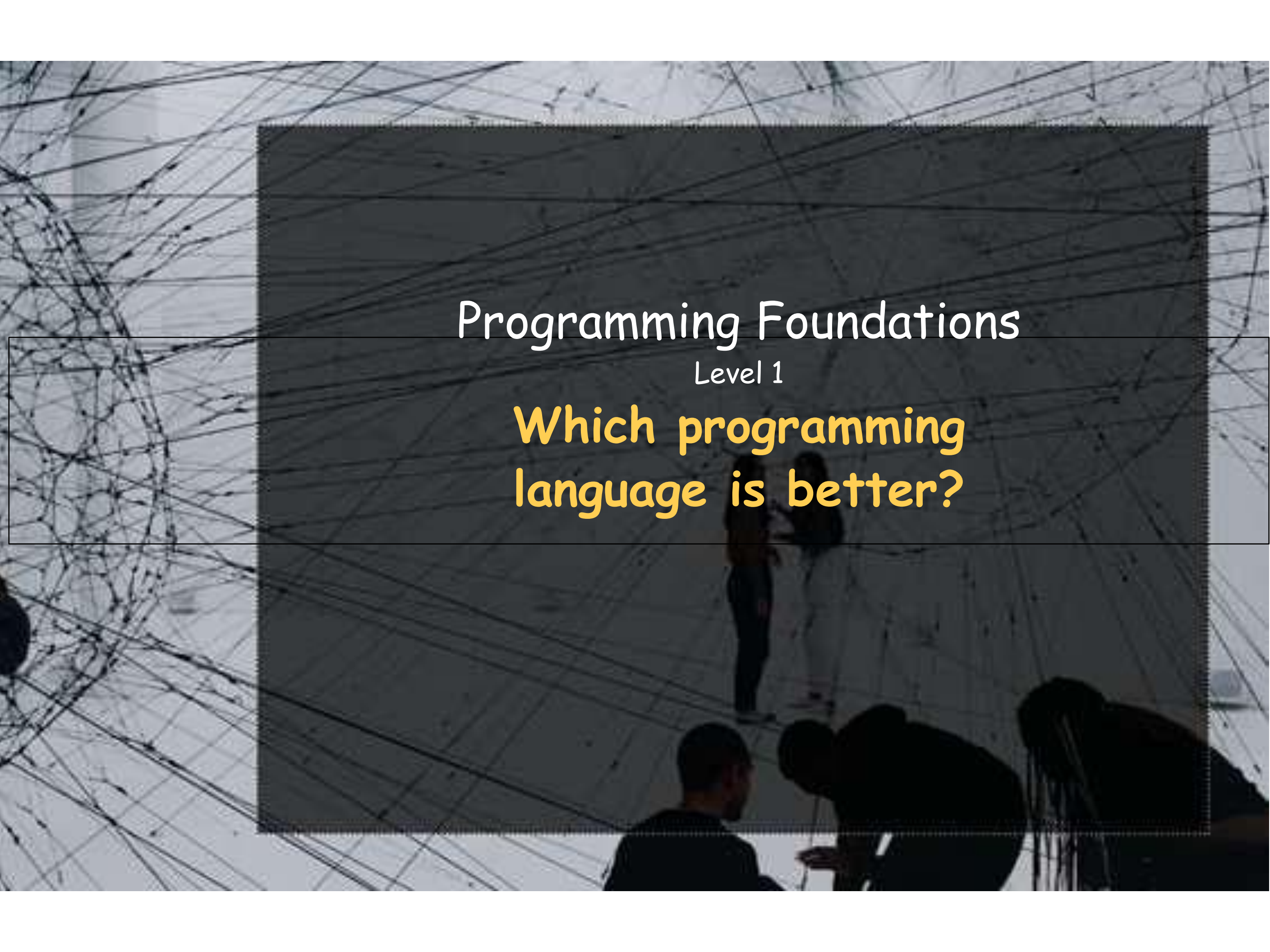




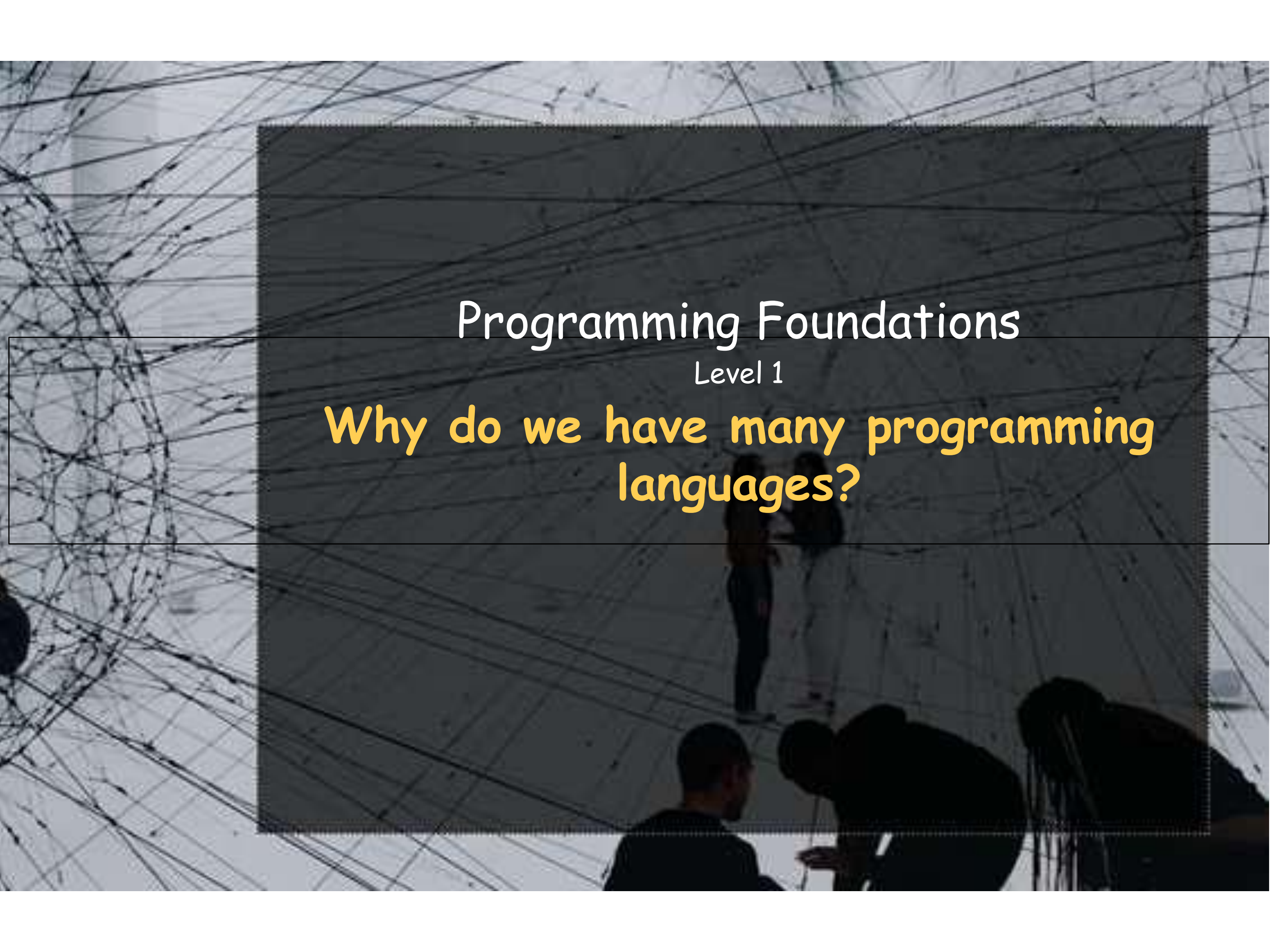
Programming Foundations Level 1

The background of the slide features a complex, abstract pattern of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, the silhouettes of several people are visible, appearing to be in a collaborative setting, possibly a meeting or a workshop, with some individuals leaning over a table or screen.

Programming Foundations

Level 1

**Which programming
language is better?**

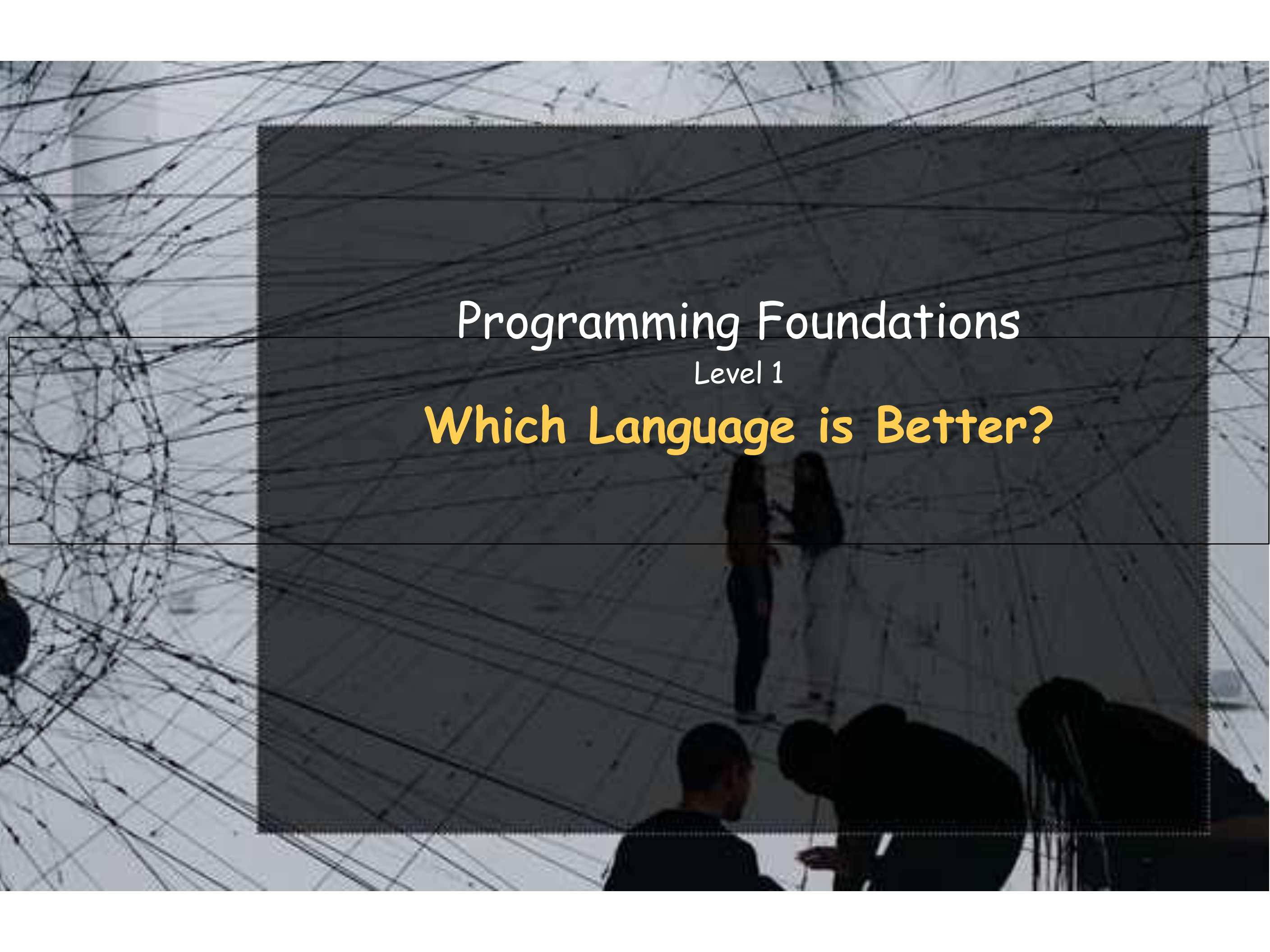
The background of the slide features a complex, abstract pattern of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, there are dark silhouettes of several people, some standing and some leaning over, suggesting a collaborative work environment or a public display. A large, dark, semi-transparent rectangular box is centered on the slide, serving as a backdrop for the text.

Programming Foundations

Level 1

Why do we have many programming languages?



The background of the slide features a complex, abstract pattern of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, there are dark silhouettes of several people, some standing and some leaning over, suggesting a collaborative work environment or a meeting. The overall aesthetic is technical and modern.

Programming Foundations

Level 1

Which Language is Better?

Which Programming Language Is Better? (The Right Question to Ask)

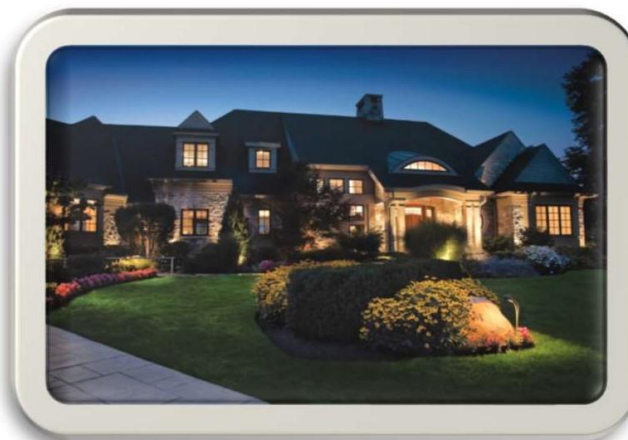
What is your best color?

This is incomplete question!!

Cars



Lights



Shirts



Which Programming Language Is Better? (The Right Question to Ask)

Asking “Which programming language is better?” is like asking “What’s the best color?” — it’s an incomplete question.

For example, I like black cars but dislike white ones, yet when it comes to clothing, I prefer white shirts and dislike black ones. So, can I really say that I hate the color white in general? Of course not!

The truth is, color alone means nothing without context — it depends on where and how it’s used. The same applies to programming languages: there’s no universally “best” language, only the best one for a specific purpose.

Driving Is Important
not the car itself!



Programming Is
Important
not the Language!

If you ask, "Which is the best car?", that's actually an **incomplete question — best for what?**

The answer will always depend on the purpose.

For example:

If you want the **best car for fuel efficiency**, I'll say **small compact cars**.

If you ask for the **best car for off-road driving**, I'll say **4x4 vehicles**.

If you want to **carry heavy loads**, I'll recommend a **truck**.

If your goal is **speed**, then it's a **Lamborghini**.

You see? There's **no single best car** — each one excels in a **different context**. You can't simply claim that a **4x4 is the best overall**, because while it's great off-road, it's **poor in fuel consumption**.

Which Language is Better?

This is incomplete question!!

It Depends on your need

You can use many programming languages at the same time.

The right answer is always **"It depends on your needs."**

You should **focus on mastering programming skills first**, because the language is only a **vehicle — not the destination**.

A programming language is simply a **tool**, and the best tool depends on the job.

Sometimes **Visual Basic** is a far better choice than **C++** for certain tasks, while in other situations **C++** clearly outperforms **Visual Basic**.

"Best for what?" Web backend? Mobile app? Embedded device? Data science?

Without context, **"Which is better?"** has **no useful answer**.

It's not about which language is superior — it's about **choosing the right one for the right situation**.

You can even **use multiple languages** on the same project.

Choose by **domain & constraints** (performance, team skills, libraries, hosting).

Many systems combine languages (e.g., **C++** core + **Python** glue + **JS/TS** frontend).



Programming Foundations

Level 1

No Racism In Programming Languages

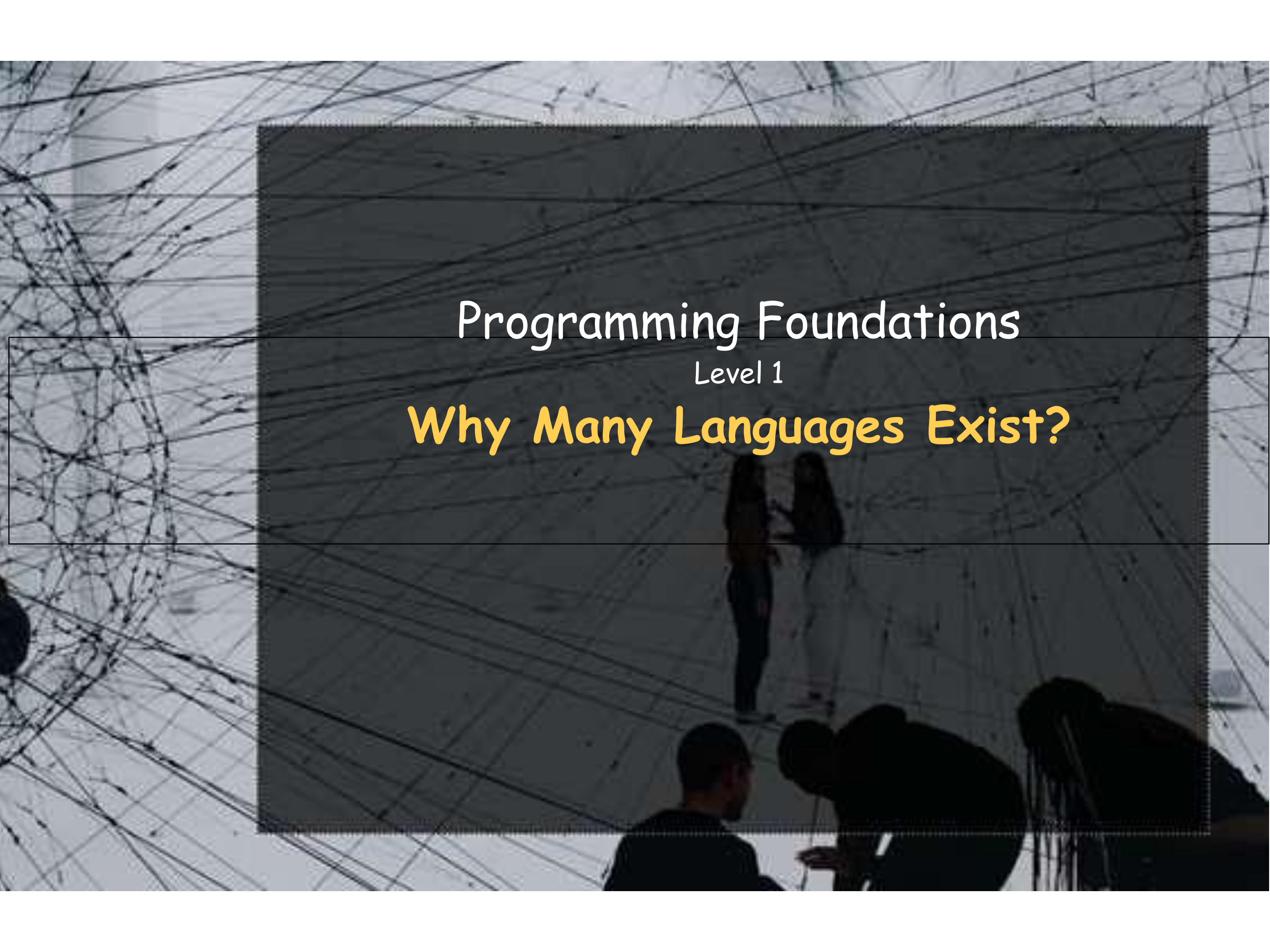


All **programming languages are great** — each one is simply a **tool in your hands** to help you achieve your goal.

Disliking or being **biased** against a language doesn't make you a stronger developer; it shows **limited understanding**. A truly skilled programmer appreciates the **value of every language** and chooses the right one for the task.

So, instead of being "language-racist," it's time to **grow as a professional** — focus on solving problems, not defending preferences.

Avoid fanboy wars. Languages are **tools**, not identities. Be **pragmatic** and respectful; use what solves the problem well.

The background of the slide features a complex, abstract network of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, there are dark silhouettes of several people, some standing and some leaning over, suggesting a collaborative work environment or a meeting. A large, dark, semi-transparent rectangular box is centered on the slide, serving as a backdrop for the text.

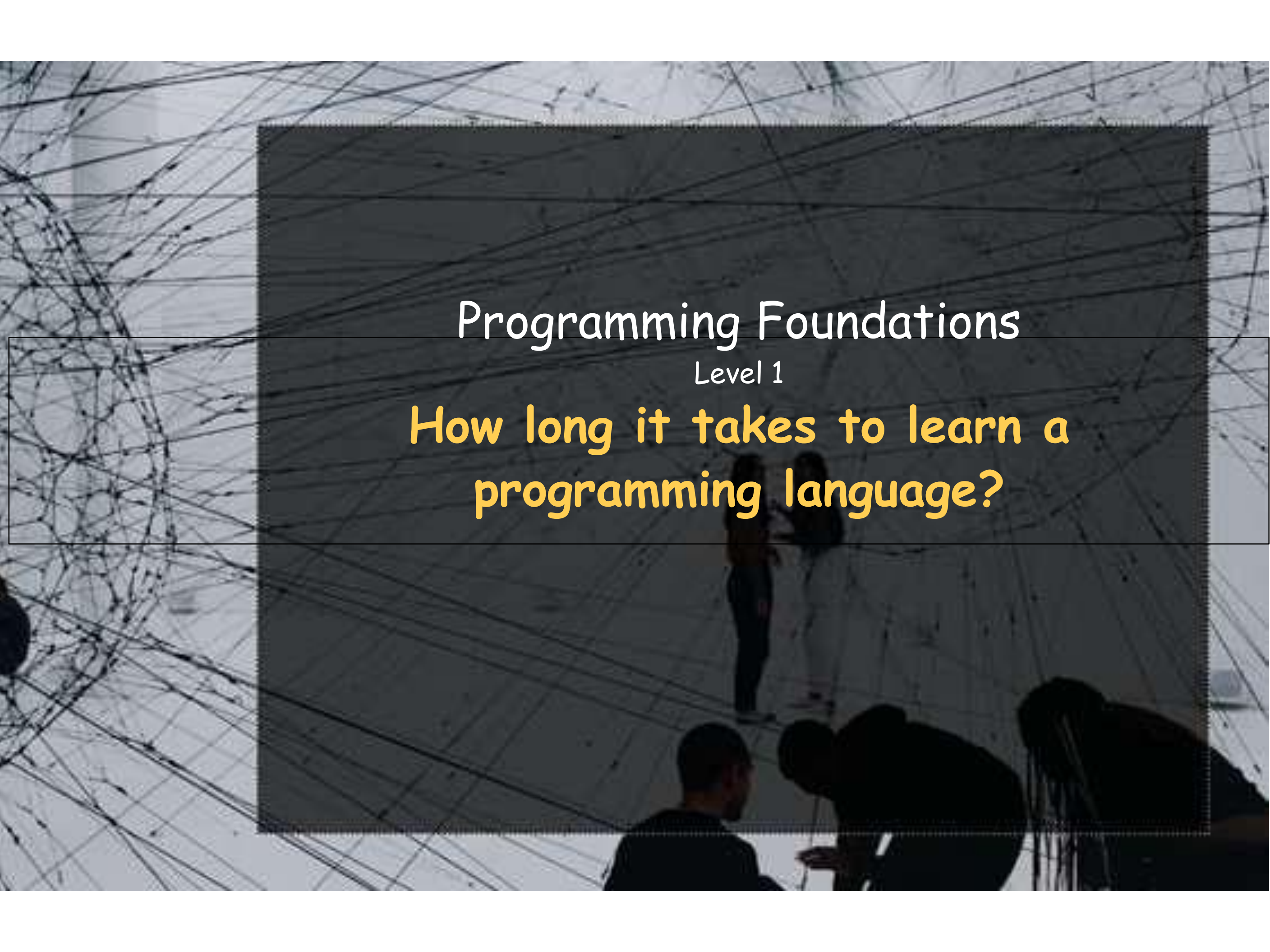
Programming Foundations

Level 1

Why Many Languages Exist?



Different problems require **different trade-offs** — such as **speed, safety, ecosystem, tooling, or domain suitability**. There's no universal language that fits every situation. Just as a craftsman uses the **right tool for each task**, a good programmer chooses the **right language for the job**.



Programming Foundations

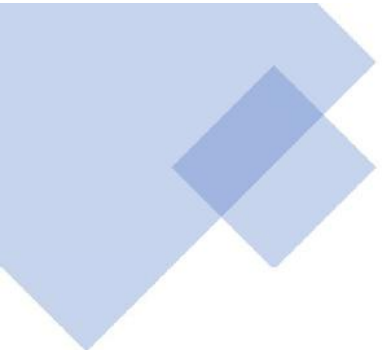
Level 1

**How long it takes to learn a
programming language?**

Less than a month's time

Learning a Programming Language Is 5% of Learn Programming

Syntax is the easy part you can learn it i a very short time. The 95% includes: Design, problem-solving, data structures, algorithms, architecture, testing, tooling, debugging, deployment, collaboration.



Understand the difference between Programming & Programming Language



Learning programming is not the same as learning a programming language — there's a big difference.

PROGRAMMING LANGUAGE I** TOOL, PROGRAMMING IS* CRAFT

THE ANALOGY: CARPENTRY



TOOL: Hammer, Saw, Scredivover
Learning the tools takes 10 minutes.

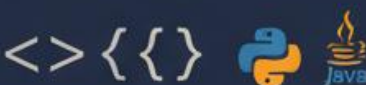


**DOES THAT MAKE YOU
A CARPENTER?**



THE CRAFT:
Building a Table.
Requires: Skill,
Practice, Creativity,
Understanding.

THE APPLICATION: PROGRAMMING



TOOL: Programming Language
(Python, Java, a JavaScript, etc.).
A Programming Language is just tool to communicate instructions.



**DOES THAT MAKE YOU
A PROGRAMMER?**



THE CRAFT:
Programming (Creating an,
a Website, a System).
Requires: Time, Practice,
Deep Logical Thinking.

LEARNING A LANGUAGE = LEARNING PROGRAMING

Focus on mastering the craft, not just collecting tools.



Imagine you want to become a carpenter.

You must learn how to use **carpentry tools**, but more importantly, you need to master the **craft of carpentry** itself.

Tools like the **hammer, saw, and screwdriver** can be learned in just ten minutes — but does that make you a carpenter?

Of course not, because tools alone don't build a table.

Building a table requires **skill, practice, creativity, and understanding** — that's the **craft**.

Similarly, in programming:

A **programming language** is just a tool, while **programming itself** is the craft. And mastering that craft takes **time, practice, and deep logical thinking**.

A **programming language** is just a tool — **programming** is the craft.

Knowing the tools doesn't make you a carpenter; **mastering the craft** does.

You use tools to build, but you use skill to create.

Programming is not about writing code; it's about shaping logic.

- ◆ **What Is Programming?**

Programming is the art of **designing logical solutions** and expressing them precisely so that **computers can execute them**.

It's not **just about writing code** — it's about **thinking clearly** about requirements, logic, and trade-offs.

It's your ability to **break down large problems into smaller, manageable parts**, solve each one efficiently, and then **assemble them into a complete, working program** that achieves the intended goal.

- ◆ **What Is a Programming Language?**

A **programming language** is simply a **tool** — a way to **communicate instructions** to a computer.

It provides the **syntax and structure** that programmers use to express their ideas and solutions.

Just like a **hammer or saw** helps a carpenter build furniture, a programming language helps a programmer **build software**.

The true power lies not in the tool itself, but in **how skillfully you use it**.

Driving is important, not the car itself. Likewise, **Programming is important, not the language**. Strong fundamentals transfer across languages.

DRIVING IS THE SKILL, THE CAR JUST A TOOL

THE ANALOGY: DRIVING



TOOL: The Car (Ford, Toyota, Tesla, etc.).



**DOES OWNING A CAR
MAKE YOU
GOOD DRIVER?**



THE SKILL: Driving Safely
Efficiently.
Requires: Coordination,
Experience,
Decision-Making.

THE APPLICATION: PROGRAMMING



TOOL: The Programming Language
(Python, Java, C++, etc). A
Programming Language is just tool to communicate
instructions.



**DOES LEARNING A LANGUAGE
MAKE YOU
PROGRAMMER?**



THE SKILL: Programming
(Solving Problems,
Creating Solutions).
Requires: Logic, Practice,
Deep Logical Thinking.

STRONG FUNDAMENTALS TRANSFER ACROSS LANGUAGES

Focus on the skill, not the craft, not just collecting tools.

💡 START SMART — DON'T BEGIN WITH MODERN LANGUAGES



THE ANALALGY: DRIVING

MODERN CAR WITH PARK ASSIT



DOES THAT MAKE YOU A SKILLED DRIVER?
✗ No! Without tools, iou'd struggle.



MANUAL PARKING



- Mirrors,
- Judgment, Control

THE APPLICATION: PROGRAMMING

MODERN LANGUAGES (Python, JavaScript, etc.).



Illusion of being a good programmer



ARE YOU BUILDING REAL SKILLS?
✗ No! Ready-made functions are a crutch.

C++

START WITH C++

- Easy & Powerful
- 10/10 Fundamentals
- Teaches Core Concepts

MASTER BASICS → LEARN ANY MODERN LANGUAGE EASILY

BUILD FUNDAMENTALS FIRST

Use modern tools as a helpful feature, not a crutch.



Never start learning with **modern programming languages**, because they are full of **ready-made functions and tools** that can give you the **illusion** that you're a good programmer — when you're actually not building the real skills yet.

Think of it like **learning to drive**:

The hardest part for beginners is **reversing**.

Now imagine I teach you using a **modern car** with **park assist, cameras, and sensors**.

You'll park perfectly every time — but does that make you a skilled driver?

✗ No! Because without those tools, you'd struggle.

To become a **real driver**, you must first learn **manual parking** — using **mirrors, judgment, and control**. Once you master that, you can use parking assistants as a **helpful feature**, not a crutch.

The same applies to programming.

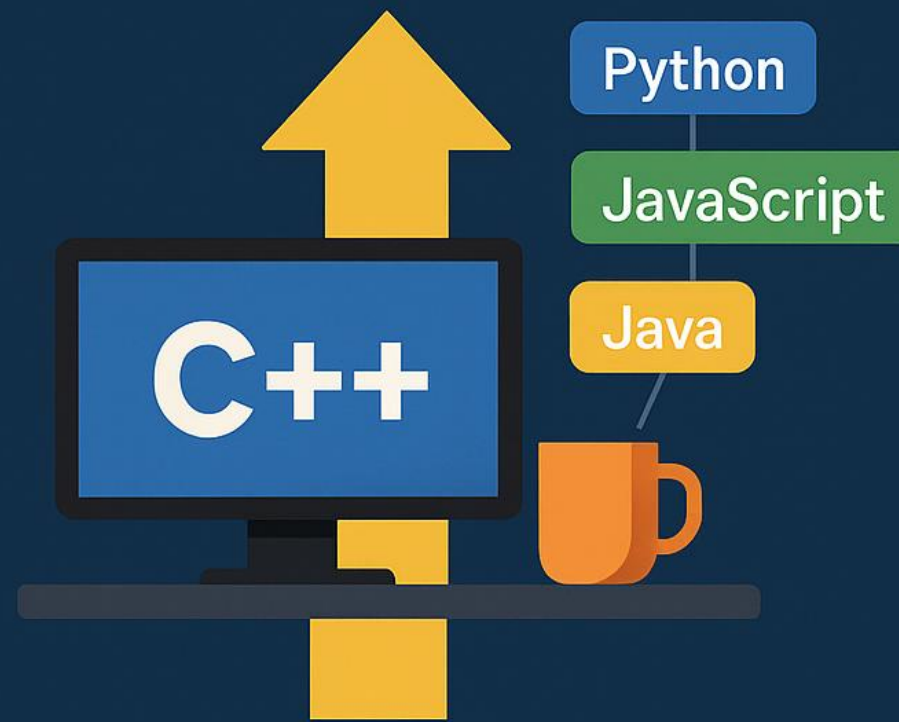
Start with **C++** — it's actually an **easy and powerful language** that teaches you **all the core programming concepts (10/10 fundamentals)**.

Once you master those basics, you can **learn any modern language easily** at any time. and this is not wasting your time because learning any other languages will not take time at all.

C++ = Salary ++

– Even If You Don't Use It at Work!

- Build strong programming fundamentals
- Understand computers deeply (memory, performance, low-level logic)
- Learn any language fast



Master the craft. The tools will follow.

Master the craft. The tools will follow.

Learning **C++** is one of the smartest investments you can make as a programmer.

Even if you don't use it in your daily job, it will:

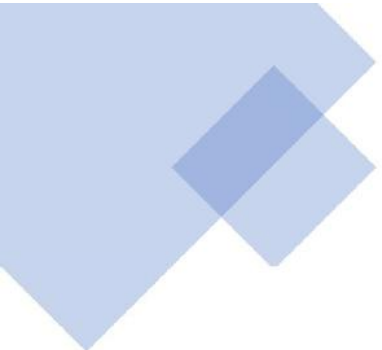
- Build **strong programming fundamentals**.

- Teach you how computers truly work — memory, logic, performance.

- Give you the **key to learn any other programming language** quickly and easily.

- Easy to learn by the way!.

C++ trains your mind to think like a **real programmer**, not just a code user — and that mindset will make you valuable in **any programming field**.



Programming

Equals

Programming Language



Programming

NOT Equals

Programming Language





Driving

Programming Languages

Are only tools !!

Mastering them alone will not
make you a good developer !!

Mastering a programming language is only

5%

of learning programming!



**DONT WASTE YOUR TIME LEARNING
MORE THAN ONE PROGRAMMING
LANGUAGE!**

You Tube

@ProgrammingAdvices
Mohammed Abu-Hadhoud

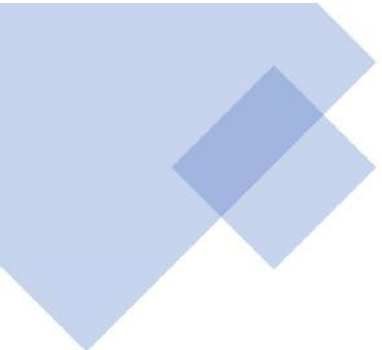
Programming Languages

***DONT WASTE YOUR TIME LEARNING
MORE THAN ONE PROGRAMMING
LANGUAGE!***

**If you master one language you
can learn any other language in
less than a month's time**



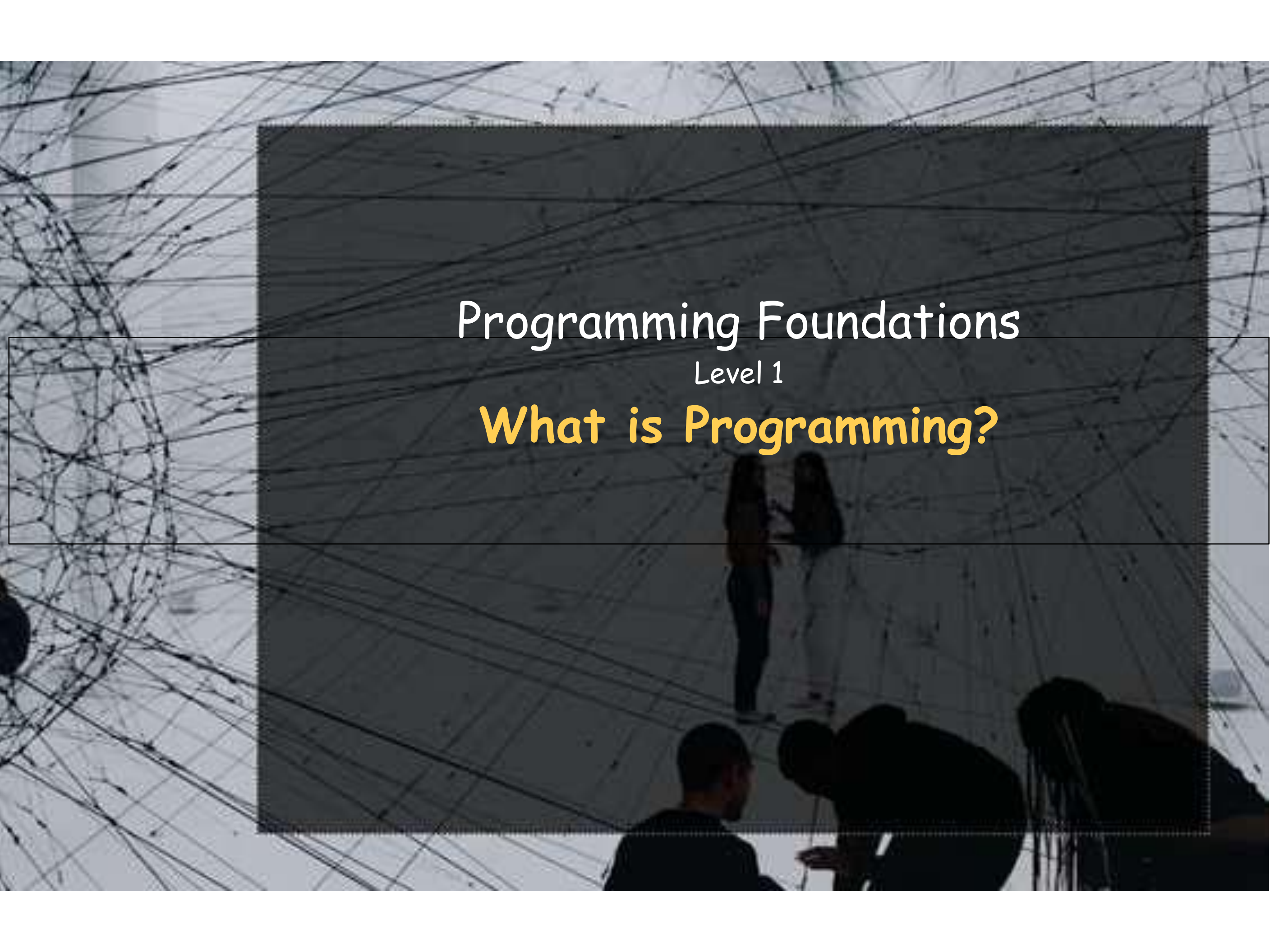
**All programming languages share the same concepts,
the only difference is Syntax/Tools.**



Focus on learning
"Programming"

Not Programming languages!



The background of the slide features a complex, abstract network of thin, dark lines crisscrossing a light gray background, resembling a neural network or a web. In the lower portion of the image, there are dark silhouettes of several people, some standing and some leaning over, suggesting a collaborative work environment or a classroom setting. A large, dark gray rectangular area is overlaid on the right side of the image, serving as a backdrop for the text.

Programming Foundations

Level 1

What is Programming?

Programming is...

A process of creating a set of instructions that tell a computer how to perform a task.

It all depends on:

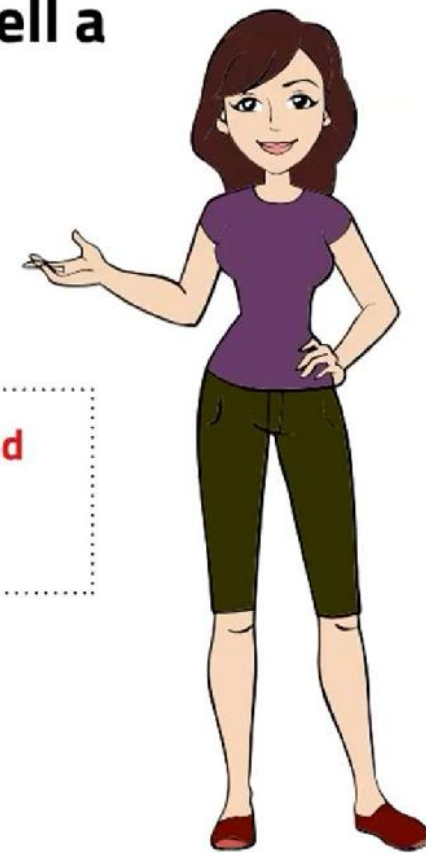
Logic

Analysis

Thinking

Ability to find solutions

Regardless of any programming language!



To make it easy for you..

Suppose that you want to be a chef



You Tube
@ProgrammingAdvices
Mohammed Abu-Hadhoud



To make it easy for you..

Suppose that you want to be a chef

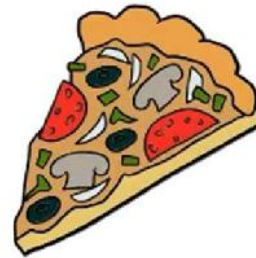
**You spend all your time
to learn how to cook**



You Tube
@ProgrammingAdvices
Mohammed Abu-Hadhoud



If you want to cook Pizza



There are steps and process to make it



You Tube
@ProgrammingAdvices
Mohammed Abu-Hadhoud



If you want to cook Pizza



There are steps and process to make it

Of course you need tools to help you make it



You Tube
@ProgrammingAdvices
Mohammed Abu-Hadhoud



If you want to cook Pizza



Cooking itself is
"Programming"



There are steps and process to make it

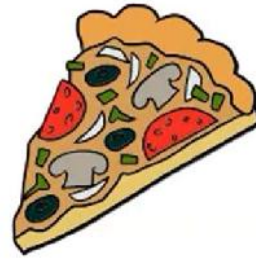
Of course you need tools to help you make it



You Tube
@ProgrammingAdvices
Mohammed Abu-Hadhoud



If you want to cook Pizza



Cooking itself is
"Programming"

There are steps and process to make it

Of course you need tools to help you make it



Tools are
"Programming Languages"



You Tube
@ProgrammingAdvices
Mohammed Abu-Hadhoud



Having all cooking tools alone



Will not make you a good chef

**You should focus to learn how to
cook rather than only buying
cooking tools**



You Tube
@ProgrammingAdvices
Mohammed Abu-Hadhoud



**Therefore mastering
programming languages alone**



Will not make you a good developer

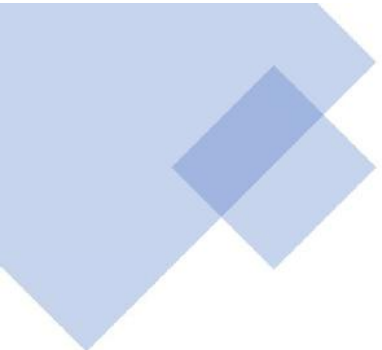
You should focus to learn

**"Programming" rather than only
learning "Programming Languages"**



You Tube
@ProgrammingAdvices
Mohammed Abu-Hadhoud





We agreed to Focus on Improving your...

Logic

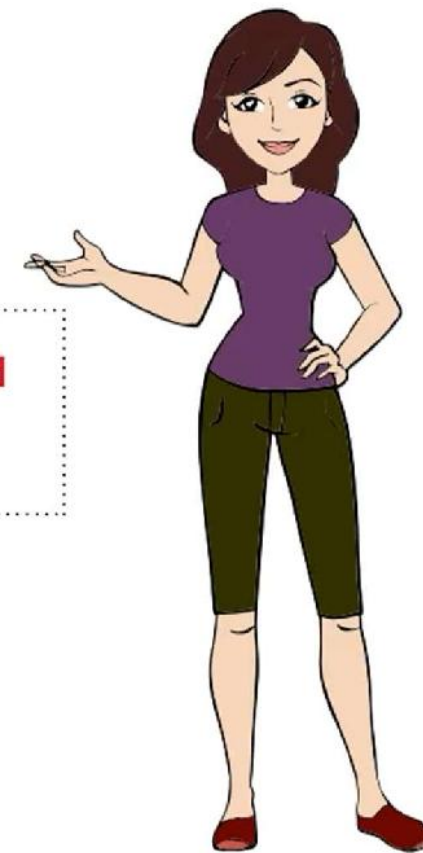
Analysis

Thinking

**Ability to find
solutions**

only using one programming language!

But how and which Language??



**Never ever start with a modern
programming language!!**



Interconnection

- ◆ “Which is better?” → “Better for what?” Context first.
- ◆ **Programming fundamentals** outlast **language trends**; language choice is **5%** of the journey.
- ◆ **Mixing languages** is normal; pick by **domain, ecosystem, and constraints**.
C++ is the key for all programming languages and it is easy.

By focusing on principles, you'll switch cars (languages) easily and drive anywhere



Activity



Instructions:

- ❶ Why is asking “Which programming language is better?” an incomplete question?
Explain using the color or car analogy.
- ❷ What is the correct answer to “Which language is better?”
What does the phrase “It depends on your needs” really mean in programming?
- ❸ Why do many programming languages exist?
List at least **three factors** that make different languages suitable for different situations (hint: speed, safety, ecosystem...).
- ❹ What's the difference between programming and a programming language?
Explain it using the carpentry analogy (tools vs craft).
- ❺ Why should you focus on learning programming fundamentals before languages?
What part of programming is the “craft,” and what part is just “tools”?
- ❻ Why is it not a good idea to start with modern languages?
Use the driving analogy (manual parking vs park assist) to explain.
- ❼ Why is C++ recommended as the first language to learn?
List three reasons based on the lesson (hint: fundamentals, concepts, mindset).
- ❽ What does “C++ = Salary ++” mean?
Explain why learning C++ adds long-term value — even if you don't use it in daily work.
- ❾ What makes a good programmer?
Is it the language they use, or something deeper?



Activity Solution



Solution:

- ① Why is asking “Which programming language is better?” an incomplete question?

Explain using the color or car analogy.

Because “better” depends on **context and purpose**.

It’s like asking “What’s the best color?” or “What’s the best car?” — it makes no sense without specifying what it’s used for.

Example: a **4x4** is great off-road but bad for fuel efficiency, just like some languages are fast but not easy to use.

- ② What is the correct answer to “Which language is better?”

What does the phrase “It depends on your needs” really mean in programming?

The correct answer is **“It depends on your needs.”**

Each language is designed to solve certain problems efficiently.

There is no single “best” language — only the **right language for a specific purpose**.

3 Why do many programming languages exist?

List at least three factors that make different languages suitable for different situations (hint: speed, safety, ecosystem...).

Because different problems require different trade-offs, such as:

Speed & performance (C++, Rust)

Safety & reliability (Java, Ada)

Ecosystem & libraries (Python, JavaScript)

Tooling or platform compatibility (C#, Swift)

Domain fit (SQL for databases, HTML for web pages)

Just like tools in a toolbox — each one is specialized for a certain job.

4 What's the difference between programming and a programming language?

Explain it using the carpentry analogy (tools vs craft).

A **programming language** is just a **tool** to express instructions, while **programming** is the **craft of problem-solving**.

Learning syntax is easy — like learning to use a hammer or saw.

Becoming a programmer means mastering how to design, plan, and combine those tools to build complete solutions — like crafting a perfect piece of furniture.

5 Why should you focus on learning programming fundamentals before languages?

What part of programming is the “craft,” and what part is just “tools”?

Because the fundamentals — **logic, algorithms, data structures, and problem-solving** — are what make you a real programmer.

Once you understand those concepts, you can switch between any language easily.

Languages change, but the **thinking skills** never go out of date.

6 Why is it not a good idea to start with modern languages?

Use the driving analogy (manual parking vs park assist) to explain.

Modern languages have many built-in functions and helpers that hide complexity.

They make you feel skilled, but you’re relying on the tools — not your understanding.

It’s like learning to drive using **automatic parking sensors** — you’ll park perfectly, but without them, you’ll struggle.

Start with **C++**, which forces you to think logically and learn the true mechanics of programming.

7 Why is C++ recommended as the first language to learn?

List three reasons based on the lesson (hint: fundamentals, concepts, mindset).

Because C++ teaches you all the fundamentals (10/10 concepts).

It's:

Powerful yet easy to learn once guided correctly.

Helps you understand how **computers truly work** (memory, logic, speed).

Gives you the **foundation** to learn any modern language quickly.

Once you master C++, every other language feels simple.

8 What does "C++ = Salary ++" mean?

Explain why learning C++ adds long-term value — even if you don't use it in daily work.

It means learning C++ increases your **value and career potential**.

Even if you don't use it daily, it builds **strong fundamentals** that make you adaptable and capable in any programming field.

C++ teaches **discipline, structure, and performance thinking** — qualities that employers reward highly.

9 What makes a good programmer?

Is it the language they use, or something deeper?

A good programmer is defined by how they think, analyze, and solve **problems**, not by which language they use.

Programming is about **logic, design, and creativity** — the language is just the medium used to express those ideas.

✓ Final Summary:

There's no "best language" — **only the best for the task.**

Focus on **programming concepts**, not syntax.

C++ builds fundamentals that unlock every other language.

Languages are **tools** — programmers are the **craftsmen** who use them to build solutions.

🏁 Conclusion

- ✅ "Which language is better?" → **Clarify the goal first.**
- ✅ Prioritize **programming fundamentals**; languages are **interchangeable tools**.
- ✅ Choose per **domain fit, ecosystem, performance, team, and market** — and don't be afraid to **mix languages**.

This mindset keeps you flexible, effective, and employable across fast-changing tech stacks.

Thank you 😊